

Codebase Documentation and Scalability Report

Project: nutrition-plan-app

Generated from local code inspection only. No code was modified or deleted.

A. Executive Summary

This project is a Node.js/Express nutrition planning web app called Pinch. It serves static frontend pages from `public/`, exposes JSON APIs from `src/routes/`, stores users/customers/folders/plans in PostgreSQL through Sequelize models, uses Firebase Authentication to verify browser sign-ins, issues its own application JWT session cookie, generates meal plans from local JSON nutrition data, and exports saved plans as PDFs through Puppeteer.

The main runtime path is:

1. `src/server.js` loads environment variables, authenticates the database, and starts `src/app.js`.
2. `src/app.js` installs JSON parsing, cookie parsing, API routers, static food icons, static frontend files, page routes, and the shared error handler.
3. Browser pages under `public/` call `/api/*` endpoints.
4. Routes call controllers; controllers call repositories and services.
5. Repositories use Sequelize models or raw SQL against PostgreSQL.
6. The planner service uses `used_food_repository/foods.json` and `ready_meals/meals.json`, both cached in process memory.
7. PDF export loads saved plan data and renders HTML/CSS in a headless browser.

The largest cleanup concerns are loose historical artifacts, stale diagnostic expectations, and exported helpers that are no longer called by app code. The largest scalability risks are synchronous JSON/icon filesystem work, expensive in-process plan generation, per-request database queries without visible migration/index management, dashboard raw SQL complexity, PDF rendering with a shared Puppeteer browser, and a deployment setup that can multiply PostgreSQL connections under serverless scale.

Highest-priority improvements:

- Add database migrations and explicit indexes for user-scoped lookups.
- Add rate limiting and request validation for auth, generation, rebalance, swap, plan save, and PDF export endpoints.
- Move CPU/PDF-heavy work behind queues or background jobs before high traffic.
- Add integration tests for every API route and generation edge case.
- Clean or quarantine prototype/stale files after confirming ownership.

B. Project Structure Map

```
.
|-- api/
|   |-- index.js
|   |-- plan-export.js
|-- public/
|   |-- css/styles.css
|   |-- js/app.js
|   |-- js/auth.js
|   |-- js/customer.js
|   |-- js/dashboard.js
|   |-- customer.html
|   |-- dashboard.html
|   |-- explorer.html
|   |-- index.html
|   |-- login.html
```

```

| |-- planner.html
| |-- register.html
|-- src/
| |-- config/
| |-- controllers/
| |-- middleware/
| |-- models/
| |-- repositories/
| |-- routes/
| |-- services/
| |-- app.js
| |-- server.js
|-- icons/
|-- ready_meals/meals.json
|-- used_food_repository/foods.json
|-- ecosystem.config.cjs
|-- package.json
|-- test-solver.js
|-- vercel.json

```

`src/` is the backend application. Its organization is conventional and readable: routes map HTTP endpoints, controllers translate request/response behavior, repositories perform persistence, services implement auth helpers, nutrition logic, generation, and PDF rendering.

`public/` is the frontend. It is functional but large files such as `public/js/app.js`, `public/js/dashboard.js`, and `public/css/styles.css` should eventually be split for maintainability.

`api/` is a Vercel deployment layer. `api/index.js` delegates to the main Express app; `api/plan-export.js` isolates PDF export with serverless include files.

`used_food_repository/`, `ready_meals/`, and `icons/` are runtime data/assets. They are small enough for current use but should move toward versioned data loading/CDN behavior as traffic and dataset size grow.

Loose or historical files include `pinch-dashboard_4.html`, `pp.txt`, `prompt.txt`, `src/Pinch_UI_Polish_Report.docx`, and `USDA_database/database.zip`. These are not part of the detected runtime path.

C. Application Startup and Runtime Flow

1. Local/PM2 startup runs `node src/server.js`.
2. `src/server.js` calls `require('dotenv').config()`, imports the Express app, imports Sequelize, reads `PORT`, and calls `sequelize.authenticate()`.
3. If PostgreSQL authentication succeeds, Express listens. If it fails, the process exits with code 1.
4. Vercel startup routes all requests to `api/index.js` unless the path matches `/api/plans/:id/export.pdf`, which rewrites to `api/plan-export.js?id=:id`.
5. `src/app.js` creates the app, parses JSON up to 2 MB, parses cookies, mounts API routers, serves `/food-icons/*`, serves `public/`, serves the bundled `zxcvbn` browser file, defines page routes, then installs `errorHandler`.
6. Browser pages call `/api/auth/me` to determine session state and redirect unauthenticated users to `/login`.
7. Login/register pages load Firebase browser SDK modules from Google, fetch public Firebase config from `/api/auth/firebase-config`, sign in with Firebase, then POST the Firebase ID token to `/api/auth/session`.
8. `authController.createSession` verifies the Firebase token with Firebase Admin, syncs the app user record, signs an app JWT, and stores it in the token cookie.
9. Protected APIs use `requireAuth`, which verifies the app JWT, loads the user by ID, checks token version, and attaches `req.user`.
10. Planner calls `/api/generate-plan`, `/api/rebalance-meal`, `/api/produce-swap-options`, `/api/foods`, and `/api/preferences`.
11. Plan/customer/folder/dashboard APIs call repositories, which use Sequelize models or raw SQL.
12. PDF export loads a plan, renders HTML with the app CSS and inline export CSS, launches/reuses Puppeteer, and sends an `application/pdf` response.

D. File-by-File Documentation

.env.example

What it does: Documents the environment variable names expected by the app without containing secret values.

Input: Deployment/operator-provided environment variables.

Output/exports: No runtime exports. It communicates required configuration names such as DATABASE_URL, JWT_SECRET, JWT_EXPIRES_IN, Firebase Admin variables, Firebase web variables, and DB pool tuning variables.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Keep pool settings documented here and add production examples for serverless versus long-running processes.

.gitignore

What it does: Excludes dependencies, build outputs, local environment files, platform cache folders, and OS/editor artifacts from Git.

Input: Git status checks.

Output/exports: No runtime exports.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Add generated report/output directories if repeated audits should not enter version control.

.vercelignore

What it does: Prevents secrets, local env files, node_modules, loose outputs, and some old data folders from Vercel uploads.

Input: Vercel build/deploy packaging.

Output/exports: Deployment packaging exclusions.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Review alongside vercel.json includeFiles; mismatches can accidentally exclude runtime assets or include oversized files.

api/index.js

What it does: Serverless entry point that imports and exports the main Express app.

Input: Vercel HTTP requests rewritten to /api/index.js.

Output/exports: Exports app from src/app.js.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Because this mounts the full Express app in serverless mode, database connection pooling must be tuned for bursty concurrency.

api/plan-export.js

What it does: Dedicated serverless Express app for plan PDF export.

Input: Authenticated GET request with `id` query parameter or a URL matching `/plans/:id/export.pdf`; optional `clientName` query.

Output/exports: Exports an Express app; sends a PDF response with `Content-Type: application/pdf`, `Content-Disposition`, and `Content-Length`, or JSON errors.

Functions/classes/components/helpers:

- Inline authenticated middleware: verifies request method is GET, extracts plan ID, fetches the user-owned plan, generates PDF, and sends it.

Stay or split: Keep as one file while export is small. If PDF export gains more deployment variants, extract the shared request handler so `api/plan-export.js` and `src/controllers/planController.js` cannot diverge.

Scalability tips: PDF rendering is expensive. Add rate limiting, cache repeated exports, and consider a queued/background export path before high traffic.

ecosystem.config.cjs

What it does: PM2 process configuration for running the app as `nutrition-plan-app`.

Input: PM2 runtime.

Output/exports: Exports PM2 app config using `src/server.js`, one forked instance, `NODE_ENV=production`, `PORT`, memory restart at 350M, and timestamped logs.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: One forked instance is simple but limited. Horizontal scaling requires DB pool tuning, sticky-free sessions, and observability.

package.json

What it does: Declares package metadata, scripts, Node engine, and dependencies.

Input: npm lifecycle commands.

Output/exports: Scripts: `start`, `dev`, `check`, and `test` aliasing check.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Add separate scripts for integration tests, generation quality tests, linting, migration checks, and production smoke tests.

vercel.json

What it does: Configures Vercel rewrites and serverless function packaging.

Input: Vercel routing/build.

Output/exports: Rewrites Firebase auth helper paths, routes PDF export to `api/plan-export.js`, routes everything else to `api/index.js`, and includes required static/data files in serverless bundles.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Watch function bundle size and cold-start behavior; serverless connection storms are a risk with Sequelize unless pooling/proxying is handled.

src/server.js

What it does: Local/long-running process entry point.

Input: Environment variables, especially `DATABASE_URL` and `PORT`.

Output/exports: No exports. Starts the HTTP server after database authentication.

Functions/classes/components/helpers:

- Promise chain on `sequelize.authenticate()`: logs DB success and starts listening, or logs failure and exits.

Stay or split: Keep as one file.

Scalability tips: Add graceful shutdown, readiness/liveness endpoints that account for DB state, and centralized logging.

src/app.js

What it does: Builds the Express application.

Input: HTTP requests, cookies, JSON request bodies, static file requests.

Output/exports: Exports the configured Express app.

Functions/classes/components/helpers:

- Express app setup: installs JSON parser with 2 MB limit and `cookieParser`.
- API mounting: `/api/auth`, `/api/dashboard`, `/api/customers`, `/api/folders`, `/api/plans`, and general `/api` generation routes.
- Static serving: `/food-icons` from `icons/`, `public/` as static assets, and `/js/zxcvbn.browser.js` from `node_modules`.
- Page route handlers: serves `index.html`, `login.html`, `register.html`, `dashboard.html`, `customer.html`, `planner.html`, and `explorer.html`.
- Error middleware registration: installs `errorHandler` after routes.

Stay or split: Keep as one file for now. If middleware and page routing grow, split into `routes/pages.js`, `middleware/staticAssets.js`, and `apiRoutes.js`.

Scalability tips: Add security headers, compression, request ID logging, rate limiting, and stronger cache policy separation for hashed versus unhashed assets.

src/config/database.js

What it does: Creates the Sequelize PostgreSQL connection.

Input: `DATABASE_URL`, `DB_POOL_MAX`, `DB_POOL_MIN`, `DB_POOL_ACQUIRE_MS`, `DB_POOL_IDLE_MS`.

Output/exports: Exports `sequelize`.

Functions/classes/components/helpers:

- `envNumber(name, fallback)`: reads a numeric environment variable, validates it is finite and non-negative, otherwise returns `fallback`.

Stay or split: Keep as one file.

Scalability tips: Add different pool defaults for serverless and long-running deployments. Use `SSL validation` instead of `rejectUnauthorized: false` if provider certificates allow it.

src/config/firebaseAdmin.js

What it does: Lazily initializes Firebase Admin SDK.

Input: `FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL`, `FIREBASE_PRIVATE_KEY`.

Output/exports: `getFirebaseAdmin`, `cleanPrivateKey`.

Functions/classes/components/helpers:

- `cleanPrivateKey(value)`: trims optional wrapping quotes and converts escaped `\n` sequences into real newlines.
- `getFirebaseAdmin()`: returns existing initialized Admin SDK or initializes it from service account environment variables.

Stay or split: Keep as one file.

Scalability tips: Lazy singleton initialization is appropriate. Add startup validation in production so auth misconfiguration fails early.

src/config/nutritionConstants.js

What it does: Centralizes nutrition constants, meal distribution factors, and meal slot policies.

Input: Imported by nutrition/generation services.

Output/exports: `NUTRITION`, `MEAL_DISTRIBUTIONS`, `STANDARD_MEAL_SLOT_POLICY`, `AMBIGUOUS_MEAL_SLOT_POLICY`.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file now. Split later into `macroConstants`, `mealDistributions`, and `slotPolicies` if nutrition rules grow.

Scalability tips: Add tests around every constant-driven rule before changing values; these constants affect every generated plan.

src/config/preferenceTaxonomy.js

What it does: Defines preference/allergen/category taxonomy and converts food data into UI preference options.

Input: Food records from `loadFoods()` and raw preference terms from user input.

Output/exports: `getPreferenceOptions`, `normalizeToken`, `resolvePreferenceTerms`.

Functions/classes/components/helpers:

- `getPreferenceOptions(foods)`: returns avoid-food preference options grouped from taxonomy and food data.
- `buildCategoryOptions(foods)`: creates canonical category options from food categories.
- `preferenceOptionSort(a, b)`: sorts categories before allergens and labels alphabetically.

- `resolvePreferenceTerms(terms)`: maps raw food/category/taxonomy terms into selected food IDs, semantic tags, and unknown terms.
- `findTaxonomyEntry(normalizedTerm)`: finds taxonomy entry by normalized ID, label, or alias.
- `normalizeToken(value)`: lowercases, trims, and normalizes underscores/hyphens/spaces.
- `humanizeCategory(value)`: formats category identifiers for display.

Stay or split: Split soon if the taxonomy keeps expanding; separate static taxonomy data from resolution functions.

Scalability tips: Current in-memory scans are fine for 108 foods. For thousands of foods, precompute lookup maps once.

src/middleware/auth.js

What it does: Verifies application JWT sessions and attaches authenticated user context.

Input: token cookie, optional `Authorization: Bearer, JWT_SECRET`, database user record.

Output/exports: `SESSION_COOKIE_NAME`, `clearSessionCookie`, `extractBearerToken`, `requireAuth`, `sessionCookieOptions`, `verifyAppJwtRequest`.

Functions/classes/components/helpers:

- `sessionCookieOptions(maxAge)`: returns shared cookie settings.
- `clearSessionCookie(res)`: clears token and `__session` cookies.
- `extractBearerToken(req)`: reads a bearer token from the Authorization header.
- `jwtSecret()`: reads `JWT_SECRET` or throws a 500-status error.
- `verifyAppJwtRequest(req)`: verifies cookie or bearer JWT and returns decoded session payload.
- `requireAuth(req, res, next)`: validates session, loads user, checks `token_version`, attaches `req.session`, `req.firebaseUid`, and `req.user`.

Stay or split: Keep as one file.

Scalability tips: Every protected request performs a DB user lookup. Add short-lived user/session caching only after revocation semantics are preserved.

src/middleware/errorHandler.js

What it does: Converts thrown errors into JSON HTTP responses.

Input: Express error object.

Output/exports: `errorHandler`.

Functions/classes/components/helpers:

- `errorHandler(error, _req, res, _next)`: derives status and sends `{ error }`, using 500 for Sequelize errors unless status is provided.

Stay or split: Keep as one file.

Scalability tips: Add structured logging and request IDs. Avoid leaking raw internal messages for all 500 errors in production.

src/models/index.js

What it does: Imports Sequelize models and defines associations.

Input: Model definitions.

Output/exports: { User, Folder, Plan, Customer }.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file.

Scalability tips: Add migrations that match these associations and enforce indexes/foreign keys at the database layer.

src/models/User.js

What it does: Defines the user's Sequelize model.

Input: PostgreSQL user's rows.

Output/exports: User model.

Functions/classes/components/helpers: Sequelize model definition with fields for Firebase UID, email, username, password hash, names, token version, created time, and last login.

Stay or split: Keep as one file.

Scalability tips: Ensure unique indexes exist for `firebase_uid`, `email`, and `username`.

src/models/Customer.js

What it does: Defines the customer's Sequelize model.

Input: PostgreSQL customer's rows.

Output/exports: Customer model.

Functions/classes/components/helpers: Sequelize model definition with user ownership, profile fields, and timestamps.

Stay or split: Keep as one file.

Scalability tips: Add composite indexes on `(user_id, id)`, `(user_id, lower(trim(name)))`, and optionally `(user_id, updated_at)`.

src/models/Folder.js

What it does: Defines the folder's Sequelize model.

Input: PostgreSQL folder's rows.

Output/exports: Folder model.

Functions/classes/components/helpers: Sequelize model definition with user owner, parent folder, name, and created timestamp.

Stay or split: Keep as one file.

Scalability tips: Add indexes on `(user_id, parent_id)` and protect recursive deletion semantics.

src/models/Plan.js

What it does: Defines the plan's Sequelize model.

Input: PostgreSQL plan's rows, including JSONB `plan_data`.

Output/exports: PPlan model.

Functions/classes/components/helpers: Sequelize model definition with user/folder/customer ownership, name, JSONB plan data, active flag, last opened timestamp, and timestamps.

Stay or split: Keep as one file.

Scalability tips: Add indexes on (user_id, folder_id), (user_id, customer_id), (user_id, updated_at), (user_id, last_opened_at), and a partial/unique strategy for active customer plans.

src/routes/authRoutes.js

What it does: Registers authentication routes.

Input: HTTP requests under /api/auth.

Output/exports: Express router.

Functions/classes/components/helpers:

- GET /firebase-config: returns public Firebase browser config.
- POST /session: creates app session from Firebase ID token.
- POST /register: legacy password auth disabled response.
- POST /login: legacy password auth disabled response.
- POST /logout: clears session.
- GET /me: authenticated current-user endpoint.
- DELETE /me: authenticated account deletion.

Stay or split: Keep as one file.

Scalability tips: Rate-limit session creation, login legacy routes, and delete account.

src/routes/customerRoutes.js

What it does: Registers customer CRUD and customer-plan routes.

Input: Authenticated HTTP requests under /api/customers.

Output/exports: Express router.

Functions/classes/components/helpers:

- GET /: list customers.
- POST /: create customer.
- GET /match: match by normalized name.
- GET /:id/plans: list customer plans.
- GET /:id: fetch customer.
- PUT /:id: update customer.
- DELETE /:id: delete customer.

Stay or split: Keep as one file.

Scalability tips: Pagination/search indexes are needed before customers per user grow beyond hundreds.

src/routes/dashboardRoutes.js

What it does: Registers dashboard summary route.

Input: Authenticated GET /api/dashboard.

Output/exports: Express router.

Functions/classes/components/helpers:

- GET /: returns dashboard stats, recent plans, general plans, customers, and result limits.

Stay or split: Keep as one file.

Scalability tips: Add caching or query tuning for large users; dashboard currently runs several queries per load.

src/routes/folderRoutes.js

What it does: Registers folder tree/content and folder plan-save routes.

Input: Authenticated HTTP requests under /api/folders.

Output/exports: Express router.

Functions/classes/components/helpers:

- GET /tree: folder tree.
- GET /: root contents.
- POST /: create folder.
- GET /:id/breadcrumb: breadcrumb path.
- GET /:id: folder contents.
- PATCH /:id: rename folder.
- DELETE /:id: delete folder.
- POST /:id/plans: save a plan into a folder.

Stay or split: Keep as one file.

Scalability tips: Recursive folder behavior needs constraints and tests; deletion currently relies on database behavior for child folders/plans.

src/routes/generationRoutes.js

What it does: Registers health, reference-data, generation, rebalance, and swap APIs.

Input: HTTP requests under /api.

Output/exports: Express router.

Functions/classes/components/helpers:

- GET /health: public health check.
- GET /foods: public food list.
- GET /preferences: public preference options.
- POST /generate-plan: authenticated plan generation.

- POST /rebalance-meal: authenticated meal rebalance.
- POST /produce-swap-options: authenticated produce swap options.

Stay or split: Split soon into health/reference routes and generation action routes.

Scalability tips: Rate-limit CPU-heavy generation and rebalance endpoints. Validate request bodies with schemas.

src/routes/planRoutes.js

What it does: Registers plan CRUD, PDF export, duplicate, and activation routes.

Input: Authenticated HTTP requests under /api/plans.

Output/exports: Express router.

Functions/classes/components/helpers:

- POST /: create plan.
- GET /:id/export.pdf: export PDF.
- GET /:id: fetch plan and mark opened.
- PUT /:id: update plan.
- DELETE /:id: delete plan.
- POST /:id/duplicate: duplicate plan.
- POST /:id/active: set plan active for a customer.

Stay or split: Keep for now. Move export handler into a dedicated module if serverless and Express paths diverge.

Scalability tips: Add optimistic concurrency or updated-at checks for autosave races.

src/controllers/authController.js

What it does: Implements auth route behavior for Firebase config, session creation, logout, current user, and account deletion.

Input: Requests with Firebase ID tokens, profile body data, session user from middleware, headers for host detection.

Output/exports: createSession, deleteUserHandler, getFirebaseConfig, getMe, legacyPasswordAuthDisabled, logout.

Functions/classes/components/helpers:

- serializeUser(user): returns public user shape.
- legacyPasswordAuthDisabled(_req, res): returns HTTP 410 for old password auth endpoints.
- publicHost(req): resolves public host from forwarded or direct host headers.
- shouldUseRequestHostForAuthDomain(host): decides whether authDomain should be overridden with request host.
- jwtSecret(): reads JWT_SECRET or throws.
- signSessionToken(user): creates app JWT with user ID, Firebase UID, token version, subject, and expiry.
- getFirebaseConfig(req, res): returns Firebase public config or missing-key error.
- createSession(req, res, next): verifies Firebase ID token, checks recent auth time, syncs user, sets session cookie.
- logout(req, res, next): clears session cookies.
- getMe(req, res): returns current serialized user.
- deleteUserHandler(req, res, next): deletes app user, deletes Firebase user if present, clears cookies.

Stay or split: Keep for now. Split token/session helpers into a service if more auth providers are added.

Scalability tips: Add rate limits and structured audit logs for session and deletion actions.

src/controllers/customerController.js

What it does: Implements customer API handlers.

Input: Authenticated request user, params, query, and JSON bodies.

Output/exports: listCustomersHandler, matchCustomerHandler, createCustomerHandler, getCustomerHandler, updateCustomerHandler, getCustomerPlansHandler, deleteCustomerHandler.

Functions/classes/components/helpers:

- listCustomersHandler(req, res, next): returns up to 25 customers matching optional query.
- matchCustomerHandler(req, res, next): returns exact normalized-name match.
- createCustomerHandler(req, res, next): creates customer from request body.
- getCustomerHandler(req, res, next): fetches one customer by ID.
- updateCustomerHandler(req, res, next): updates a customer by ID.
- getCustomerPlansHandler(req, res, next): returns customer plus assigned plans.
- deleteCustomerHandler(req, res, next): removes customer and unlinks/deactivates plans.

Stay or split: Keep as one file.

Scalability tips: Add schema validation and pagination before large customer lists.

src/controllers/dashboardController.js

What it does: Implements dashboard summary endpoint.

Input: Authenticated request user and optional customerQuery, generalQuery, limit.

Output/exports: getDashboard.

Functions/classes/components/helpers:

- getDashboard(req, res, next): calls getDashboardSummary and returns dashboard JSON.

Stay or split: Keep as one file.

Scalability tips: Cache or precompute dashboard counts if users accumulate many plans.

src/controllers/folderController.js

What it does: Implements folder API behavior and folder-specific plan saving.

Input: Authenticated request user, folder params, JSON bodies.

Output/exports: getTree, getRootContentsHandler, createFolderHandler, getBreadcrumbHandler, getFolderContentsHandler, renameFolderHandler, deleteFolderHandler, savePlanInFolder.

Functions/classes/components/helpers:

- getTree(req, res, next): returns hierarchical folder tree.
- getRootContentsHandler(req, res, next): returns root folders and root plans.
- createFolderHandler(req, res, next): validates name and creates folder.

- `getBreadcrumbHandler(req, res, next)`: returns path to folder.
- `getFolderContentsHandler(req, res, next)`: returns folder, subfolders, and plans.
- `renameFolderHandler(req, res, next)`: validates name and renames folder.
- `deleteFolderHandler(req, res, next)`: deletes folder.
- `savePlanInFolder(req, res, next)`: validates folder/name/planData and creates a plan inside folder.

Stay or split: Keep as one file.

Scalability tips: Add depth limits and database constraints for folder trees.

src/controllers/generationController.js

What it does: Implements nutrition generation and reference-data API handlers.

Input: Request bodies for generation/rebalance/swap; no body for foods/preferences/health.

Output/exports: `health`, `getFoodsHandler`, `getPreferences`, `generatePlanHandler`, `rebalanceMealHandler`, `produceSwapOptionsHandler`.

Functions/classes/components/helpers:

- `health(_req, res)`: returns { status: 'ok' }.
- `getFoodsHandler(_req, res, next)`: returns normalized foods.
- `getPreferences(_req, res, next)`: returns preference options derived from foods.
- `generatePlanHandler(req, res, next)`: returns generated plan.
- `rebalanceMealHandler(req, res, next)`: validates required rebalance context and returns rebalance result.
- `produceSwapOptionsHandler(req, res, next)`: returns produce swap options.

Stay or split: Split when adding more generation actions.

Scalability tips: Add request schemas and rate limits; generation is synchronous CPU work.

src/controllers/planController.js

What it does: Implements plan CRUD, duplicate, activation, and PDF export.

Input: Authenticated user, route params, query params, JSON bodies.

Output/exports: `createPlanHandler`, `getPlan`, `exportPlanPdfHandler`, `updatePlanHandler`, `deletePlanHandler`, `duplicatePlanHandler`, `setPlanActiveHandler`.

Functions/classes/components/helpers:

- `createPlanHandler(req, res, next)`: validates name/planData and creates plan.
- `getPlan(req, res, next)`: fetches plan and marks it opened asynchronously.
- `exportPlanPdfHandler(req, res, next)`: loads plan, generates PDF, sends it.
- `updatePlanHandler(req, res, next)`: validates that at least one update field is present and updates plan.
- `deletePlanHandler(req, res, next)`: deletes plan.
- `duplicatePlanHandler(req, res, next)`: copies plan into optional target folder/name.
- `setPlanActiveHandler(req, res, next)`: marks a customer-linked plan active.

Stay or split: Split PDF export into shared service/controller before growing.

Scalability tips: Autosave needs conflict handling; PDF needs throttling.

src/repositories/customerRepository.js

What it does: Persists and queries customers and customer-linked plans.

Input: User IDs, customer IDs, names, profile bodies, touched field lists, Sequelize transaction options.

Output/exports: PROFILE_FIELD_MAP, normalizeCustomerName, cleanCustomerName, customerProfileFromInput, profileUpdatesFromTouched, findCustomerByNormalizedName, findCustomerById, resolveCustomerForPlan, createCustomer, getCustomer, updateCustomer, listCustomers, listCustomersWithPlanSummary, getCustomerPlans, deleteCustomer.

Functions/classes/components/helpers:

- normalizeCustomerName(name): trims and lowercases a name.
- cleanCustomerName(name): trims a display name.
- normalizedNameWhere(name): builds Sequelize where expression for normalized name.
- customerProfileFromInput(input): maps UI profile fields to DB columns.
- nullableNumber(value, options): converts values to finite numbers or null.
- profileUpdatesFromTouched(input, touchedFields): returns updates only for explicitly touched profile fields.
- findCustomerByNormalizedName(userId, name, options): finds one user-owned customer by normalized name.
- findCustomerById(userId, customerId, options): finds one user-owned customer by ID.
- resolveCustomerForPlan(userId, selection, planInput, options): links or creates a customer for a saved plan.
- syncTouchedProfileFields(customer, planInput, touchedFields, transaction): updates only touched profile fields.
- listCustomers(userId, options): returns up to 25 user customers, optionally name-filtered.
- listCustomersWithPlanSummary(userId): returns all user customers plus plan counts/active plans; no current app-code caller found.
- createCustomer(userId, input): validates name and inserts customer.
- getCustomer(userId, customerId): wrapper around findCustomerById.
- updateCustomer(userId, customerId, input): validates unique normalized name and updates profile.
- getCustomerPlans(userId, customerId): returns customer and summarized plans.
- deleteCustomer(userId, customerId): transactionally unlinks/deactivates plans and deletes customer.

Stay or split: Split later into profile normalization helpers and DB operations.

Scalability tips: Name matching uses SQL lower/trim expressions; add expression indexes for large customer sets.

src/repositories/dashboardRepository.js

What it does: Builds dashboard stats, customer summaries, general plan summaries, and recent plan summaries using raw SQL.

Input: User ID, optional query strings, optional limit.

Output/exports: folderPathFor, getDashboardSummary, listDashboardCustomers, listGeneralPlans.

Functions/classes/components/helpers:

- normalizeSearch(value): trims/lowercases search string.
- normalizeLimit(value): clamps limit to default 50 and max 100.
- folderPathFor(folderId, foldersById): builds folder path from in-memory folder map; no current app-code caller found.

- `folderPathFromArrays(ids, names)`: converts SQL path arrays to objects.
- `planRowToSummary(row)`: maps SQL plan row to frontend summary shape.
- `customerRowToSummary(row)`: maps SQL customer row to frontend summary shape.
- `getStats(userId)`: returns plan/customer counts and this-week counts.
- `listDashboardCustomers(userId, options)`: SQL query for customers with plan counts and active plan.
- `listGeneralPlans(userId, options)`: recursive SQL folder path query for non-customer plans.
- `listRecentPlans(userId)`: recursive SQL folder path query for last-opened plans.
- `getDashboardSummary(userId, options)`: runs stats/customers/general/recent queries in parallel and returns dashboard payload.

Stay or split: Split raw SQL queries into named files or helper builders if this grows.

Scalability tips: Recursive folder CTEs and JSONB extraction need indexes and query plans checked with realistic data.

src/repositories/folderRepository.js

What it does: Persists and queries folder hierarchy and folder contents.

Input: User IDs, folder IDs, parent IDs, names.

Output/exports: `createFolder`, `getFolderById`, `getRootContents`, `getFolderContents`, `getBreadcrumb`, `getFolderTree`, `renameFolder`, `deleteFolder`.

Functions/classes/components/helpers:

- `createFolder(userId, { name, parentId })`: validates parent and inserts folder.
- `getFolderById(folderId, userId)`: fetches user-owned folder.
- `getRootContents(userId)`: returns root folders and root plans.
- `getFolderContents(folderId, userId)`: returns folder, child folders, and plans.
- `getBreadcrumb(folderId, userId)`: walks parent links one query at a time.
- `getFolderTree(userId)`: fetches all user folders and calls `buildTree`.
- `buildTree(folders, parentId)`: recursively nests folders.
- `renameFolder(folderId, userId, name)`: updates folder name.
- `deleteFolder(folderId, userId)`: deletes folder.

Stay or split: Keep now. Consider moving tree algorithms to helper file if tree features grow.

Scalability tips: `getBreadcrumb` performs one DB query per depth; use recursive SQL for deep folder trees.

src/repositories/foodRepository.js

What it does: Loads and normalizes local food JSON and attaches icon URLs.

Input: `used_food_repository/foods.json`, matching files in `icons/`.

Output/exports: `loadFoods`.

Functions/classes/components/helpers:

- `foodIconUrlForId(id)`: checks whether `icons/{id}.png` exists and returns `/food-icons/{id}.png` or null.
- `loadFoods()`: reads food JSON once, validates array, normalizes each food, caches result.
- `normalizeFood(food)`: validates required fields and converts snake_case raw data to camelCase runtime shape.

Stay or split: Keep as one file while data remains small.

Scalability tips: `fs.existsSync` runs for every food on first load. Precompute icon existence or store icon metadata in the JSON if data grows.

src/repositories/planRepository.js

What it does: Persists and queries plans, handles customer linking, folder validation, duplication, active-plan state, and last-opened updates.

Input: User IDs, plan IDs, folder IDs, plan data JSON, names, customer selections, active flags.

Output/exports: `createPlan`, `getPlansByFolder`, `getPlanById`, `updatePlan`, `setPlanActive`, `deletePlan`, `duplicatePlan`.

Functions/classes/components/helpers:

- `stripMeta(plan)`: removes included Folder and `plan_data` from returned model JSON.
- `createPlan(userId, folderId, name, planData, options)`: transactionally validates folder, resolves customer, manages active flag, inserts plan.
- `getPlansByFolder(folderId)`: lists plans by folder without user filter; no current app-code caller found and unsafe as a public helper.
- `getPlanById(planId, userId, options)`: fetches a user-owned plan with optional Customer include and optional `async last_opened_at` update.
- `updatePlan(planId, userId, update)`: transactionally updates plan fields, resolves customer, and enforces active-plan rules.
- `setPlanActive(planId, userId)`: marks a customer-linked plan active and unsets other active plans for that customer.
- `unsetActivePlansForCustomer(userId, customerId, transaction, exceptPlanId)`: validates customer and deactivates other active plans.
- `deletePlan(planId, userId)`: deletes a user-owned plan.
- `duplicatePlan(planId, userId, targetFolderId, newName)`: copies a plan with inactive state and no last-opened timestamp.

Stay or split: Split active-plan/customer-linking behavior if more plan states are added.

Scalability tips: Add DB-level constraint for one active plan per customer; do not rely only on transaction logic.

src/repositories/readyMealRepository.js

What it does: Loads ready-meal bundle data and converts each meal into ingredient components.

Input: `ready_meals/meals.json`.

Output/exports: `loadReadyMealBundles`.

Functions/classes/components/helpers:

- `loadReadyMealBundles()`: reads JSON once, flattens `meal_bundles` by tag, maps component slots, applies aliases, caches, and validates.
- `validateReadyMeals(meals)`: requires non-empty data, unique IDs, meal tags, and at least one usable ingredient.

Stay or split: Keep as one file.

Scalability tips: Validate data in CI rather than only at runtime startup/first use.

src/repositories/userRepository.js

What it does: Persists and queries user records and syncs Firebase identities into application users.

Input: User IDs, Firebase UUIDs, emails, Firebase-derived profiles, transactions.

Output/exports: `findUserById`, `findUserByFirebaseUid`, `findUserByEmail`, `updateLastLogin`, `incrementTokenVersion`, `deleteUser`, `syncFirebaseUser`, `usernameBaseFromSeed`.

Functions/classes/components/helpers:

- `findUserById(id)`: fetches public user fields by primary key.
- `findUserByFirebaseUid(firebaseUid)`: fetches by Firebase UID; no current app-code caller found.
- `findUserByEmail(email)`: fetches by normalized email; no current app-code caller found.
- `reloadPublicUser(user, transaction)`: reloads public attributes inside a transaction.
- `updateLastLogin(id)`: updates login timestamp; no current app-code caller found outside sync path.
- `incrementTokenVersion(id)`: increments token version; no current app-code caller found.
- `deleteUser(id)`: deletes user row.
- `usernameBaseFromSeed(seed)`: builds sanitized username base.
- `buildUniqueUsername(seed, transaction)`: tries base and suffixes to avoid username collision.
- `syncFirebaseUser(profile)`: transactionally creates or links app user from Firebase profile and updates last login.

Stay or split: Split username generation and Firebase sync if account features grow.

Scalability tips: Username uniqueness loop can perform many sequential queries under collisions; use DB retry-on-conflict for very large systems.

src/services/firebaseAuthService.js

What it does: Provides Firebase auth normalization, validation, and public config helpers.

Input: Firebase decoded tokens, client profile object, environment variables.

Output/exports: `MAX_SESSION_MS`, `assertFirebaseTokenCanAccessApp`, `cleanPrivateKey`, `isEmailPasswordProvider`, `normalizeEmail`, `profileFromFirebaseToken`, `publicFirebaseConfigFromEnv`, `splitDisplayName`.

Functions/classes/components/helpers:

- `normalizeEmail(email)`: trims/lowercases valid strings or returns null.
- `splitDisplayName(displayName, email)`: derives first/last names from display name or email local part.
- `limitName(value)`: trims and caps names at 50 chars.
- `capitalize(value)`: capitalizes first character.
- `providerId(decodedToken)`: returns Firebase sign-in provider.
- `isEmailPasswordProvider(decodedToken)`: checks provider is password.
- `assertFirebaseTokenCanAccessApp(decodedToken)`: requires UID and verified email for password provider.
- `profileFromFirebaseToken(decodedToken, clientProfile)`: builds app profile from Firebase token/client data.
- `publicFirebaseConfigFromEnv(env)`: returns browser Firebase config plus missing required keys.

Stay or split: Keep as one file.

Scalability tips: Auth checks are lightweight; main scale concern is downstream user sync and per-request user lookup.

src/services/nutritionService.js

What it does: Calculates BMR, maintenance, calorie goals, macro targets, meal splits, meal macro windows, and shared macro utilities.

Input: Normalized nutrition input and constants.

Output/exports: NUTRITION, buildMealTargets, calculateBmr, calculateDailyTargets, calculateDailyMacroRanges, calculateGoalCalories, calculateMacroTargets, calculateNutritionDetails, buildScaledMealMacroWindows, distributeMacrosAcrossMeals, getMealSlotProfile, macrosForFoodPortion, maintenanceCalories, roundToNearest, scaleTargets, splitMeals, sumTargets, clamp.

Functions/classes/components/helpers:

- `calculateBmr(input)`: applies Mifflin-St Jeor formula.
- `maintenanceCalories(input)`: multiplies BMR by activity multiplier.
- `calculateGoalCalories(input, maintenance)`: applies maintain/gain/loss target and sex-specific calorie floor.
- `withCalorieFloor(goalCalories, extras)`: nested helper returning target/floor metadata.
- `calculateMacroTargets(input, targetCalories)`: calculates protein/fat grams and remaining carbs.
- `calculateDailyMacroRanges(weightKg, targetCalories)`: builds daily macro ranges.
- `calculateNutritionDetails(input)`: returns full nutrition calculation and targets.
- `calculateDailyTargets(input)`: returns only target macros.
- `splitMeals(dailyTargets, input)`: wrapper for buildMealTargets.
- `buildMealTargets(dailyTargets, input)`: creates per-meal targets and macro windows.
- `getMealSlotProfile(numberOfMeals, distribution)`: selects distribution and slot policy.
- `buildSlotProfile(args)`: builds one slot profile with calorie bounds and macro ratio range.
- `fixedMacroRatioRangeFor(profileTag)`: reads configured ratio range.
- `buildScaledMealMacroWindows(dailyTargets, profiles)`: scales raw per-slot protein/fat windows to daily ranges.
- `carbRangeFromCaloriesProteinFat(calories, proteinG, fatG)`: derives carb range from calorie/protein/fat constraints.
- `rawMacroWindowFor(profile, dailyCalories, macro)`: converts macro calorie ratios into grams.
- `scaleRawWindows(rawWindows, dailyRange)`: scales raw windows to daily min/max.
- `distributeMacrosAcrossMeals(dailyTargets, profiles)`: distributes target macro calories across slots.
- `distributeMacroCaloriesAcrossMeals(profiles, rowTotals, macro, totalMacroCalories)`: normalizes macro calorie allocation by ratio midpoints.
- `scaleTargets(targets, factor)`: multiplies target totals by factor; no current app-code caller found.
- `ratioMidpoint(rangeOrValue)`: returns numeric value or min/max midpoint.
- `macrosForFoodPortion(food, quantityG)`: converts food per-100g rates into portion totals.
- `sumTargets(values)`: sums macro target objects.
- `roundToNearest(value, step)`: rounds to nearest step.
- `clamp(value, min, max)`: clamps a number.
- `sum(values)`: sums numeric array.

Stay or split: Split soon. This file is readable but dense; separate energy formulas, meal slot/window logic, and macro utilities.

Scalability tips: Pure CPU math is fine. Protect correctness with tests because small changes affect all plans.

src/services/planGenerator.js

What it does: Generates plans from ready-meal bundles, filters foods by preferences, diagnoses impossible plans, solves portion grids, rebalances edited meals, and produces produce swap options.

Input: Raw planner input, local foods, local ready meals, rebalance/swap request bodies.

Output/exports: generatePlan, getFoods, rebalanceMeal, getProduceSwapOptions.

Functions/classes/components/helpers:

- `getFoods()`: returns cached normalized foods.
- `generatePlan(rawInput)`: public wrapper around internal generation.
- `_generatePlanInternal(rawInput)`: normalizes input, calculates targets, filters foods, generates meals, builds diagnostics, and shapes response.
- `debugOptimizer(message, payload)`: conditional diagnostic logging.
- `totalsForMeals(meals)`: sums meal totals.
- `calculateResidual(dayTotals, dailyTarget)`: returns target-minus-actual residuals.
- `computeDailyPlanBounds(dailyTarget)`: builds daily calorie/protein/fat bounds.
- `residualTolerances(dailyTarget)`: returns allowed daily residual amounts.
- `calculateResidualScore(dayTotals, dailyTarget, tolerances)`: scores daily fit against bounds and closeness.
- `planBoundsViolationAmount(total, bounds)`: returns amount outside bounds.
- `residualWithinTolerance(dayTotals, dailyTarget, tolerances)`: checks all daily bounds.
- `dailyTotalsWithinPlanBounds(dayTotals, dailyTarget)`: alias for daily tolerance check.
- `buildPlanDiagnostics(dayTotals, dailyTarget, meals)`: classifies pass/warning/error and returns residual/bounds/totals diagnostics.
- `isRequiredMainSlot(meal)`: treats non-snack meals as required.
- `calculateResidualPercent(dayTotals, dailyTarget)`: returns percentage residuals or null for zero targets.
- `structuralCauseFor(key, residualValue, meals)`: explains missing-template or serving-bound causes.
- `normalizeInput(input)`: validates and normalizes body stats, goal, diet, meal count, preferences, and Ramadan flag.
- `normalizeList(value)`: normalizes array/comma-separated preference values.
- `filterFoods(foods, input)`: applies diet, allergen, dislike, and avoid-food restrictions.
- `searchableTermsForFood(food)`: returns normalized terms for matching.
- `hasSemanticMatch(foodTerms, selectedTerms)`: checks term overlap.
- `servingRealismPenalty(items)`: scores portion realism.
- `macroFitDetails(items, target)`: returns totals, residual, tolerance flag, and score.
- `roundedMacros(macros)`: rounds macro object values for diagnostics.
- `generateReadyMealDay(args)`: builds candidate sets and selects best daily combination, or empty failed meals.
- `readyMealCandidatesForMeal(args)`: solves and ranks ready meals for one meal target.
- `selectReadyMealDayCombination(candidateSets, dailyTargets)`: beam-searches candidate combinations.
- `compareDayCandidates(a, b, dailyTargets)`: ranks daily candidates.
- `daySignature(day)`: stable candidate ID signature.
- `buildReadyMealFromCandidate(args)`: creates meal object from selected candidate.
- `readyMealGenerationDebug(args)`: builds debug payload for successful candidate.
- `solveReadyMealCandidate(readyMeal, allowedFoodByName, target, options)`: maps ingredients, searches feasible portions, and returns candidate.
- `buildReadyMeal(args)`: builds final meal response object.
- `buildReadyMealOptions(args)`: maps alternates into option response objects.
- `hydrateSolvedItems(items)`: adds alternatives arrays and totals to solved items.
- `readyMealDisplayName(readyMeal)`: formats ready meal display name.
- `normalizeIngredientName(name)`: trims/lowercases ingredient name.

- `logMealGenerationFailure(args)`: builds debug payload for failed slot.
- `mealOptionSignature(items)`: joins food IDs for option uniqueness.
- `isStrictMealOptionFit(totals, target)`: checks meal tolerance.
- `mealOptionForTarget(option, target)`: normalizes and validates one meal option.
- `emptyMeal(target, reason, generationDebug)`: returns empty failed meal response.
- `templateTagsForMealTag(mealTag)`: maps special slots to ready-meal tags.
- `clampServing(food, quantityG)`: clamps/rounds serving within food bounds.
- `roundServingWithinBounds(quantityG, min, max, step)`: clamps and rounds quantity.
- `totalsForItems(items)`: sums item totals.
- `computeMealBounds(target)`: returns macro windows or throws.
- `cloneMacroBounds(bounds)`: copies macro bounds.
- `validateMealSwap(args)`: validates proposed totals against meal bounds.
- `findBoundsViolation(totals, bounds)`: returns first violation.
- `findBoundsViolations(totals, bounds)`: returns all calorie/protein/fat violations.
- `isWithinTolerance(items, target)`: checks item totals against target.
- `targetToleranceBounds(target)`: wrapper for meal bounds.
- `totalsWithinMealTolerance(totals, target)`: checks totals against target bounds.
- `mealScore(items, target)`: scores calorie/protein/fat closeness.
- `produceGroup(food)`: categorizes food as fruit/vegetable or null.
- `resolveMealActionItems(rawItems)`: converts request items to full food objects.
- `resolveFoodForMealAction(item, foodMap)`: resolves known or custom food.
- `dominantMacroRole(macros)`: classifies custom food by dominant macro calories.
- `findBestPortionGridFit(items, target, bounds, seedItems, options)`: recursive bounded portion search.
- `canStillFit(totals, pos)`: nested pruning helper.
- `lowerBoundScore(totals, pos)`: nested optimistic score helper.
- `visit(pos, totals)`: nested recursive search helper.
- `macroBoundKeys(bounds)`: selects bounded macro keys.
- `servingGridCandidates(food, seedQuantityG, step)`: returns possible serving quantities.
- `servingStepCount(min, max, step)`: counts grid positions.
- `macroLeverage(variable, keys)`: estimates adjustable macro impact.
- `feasibleQuantitiesForVariable(variable, totals, remaining, bounds, keys, step)`: prunes quantity candidates.
- `macrosForRates(rates, quantityG)`: calculates macros from rates.
- `emptyMacroRange()`: returns zero min/max macro range.
- `variableMacroRange(variable)`: returns min/max macros for variable.
- `addMacroRanges(left, right)`: adds min/max ranges.
- `addMacros(left, right)`: adds macro totals.
- `subtractMacros(left, right)`: subtracts macro totals.
- `macroBoundFitScore(totals, target)`: scores fit against macro windows or target.
- `compareRankedMealCandidates(a, b, target)`: ranks meal candidates.
- `mealRankTuple(totals, target, bounds)`: returns tuple used for sorting.
- `rangeMidpoint(range)`: returns midpoint.
- `rebalanceMeal(args)`: validates edited meal, tries changed-item fit, whole-meal fit, or returns failure.

- `rebalanceSuccess(items, mealTarget, dailyContext, totals, fitSource)`: shapes successful rebalance response after validation.
- `findChangedItemOnlyFit(items, changedItemIndex, target, bounds)`: solves by changing only one item.
- `findWholeMealDistributionFit(items, target, bounds)`: solves by redistributing all item quantities.
- `getProduceSwapOptions(args)`: finds produce replacements that can rebalance into target.
- `hydrateProduceSwapItems(requestItems, solvedItems)`: converts solved IDs back to food/item objects.

Stay or split: Split this file. Suggested modules: input validation, preference filtering, ready-meal candidate generation, diagnostics, portion solver, rebalance API, produce swaps, and macro utilities.

Scalability tips: This is CPU-heavy and synchronous. Add request limits, memoization for repeated candidate sets, worker threads or background jobs, and performance tests with larger food/template datasets.

src/services/planPdfService.js

What it does: Generates plan export PDFs by rendering HTML/CSS in Puppeteer.

Input: Plan record with `plan_data`, optional `clientName`, app CSS, icon files.

Output/exports: `generatePlanPdf`, `pdfFilename`.

Functions/classes/components/helpers:

- `generatePlanPdf(plan, options)`: wraps render in timeout.
- `renderPlanPdf(plan, options)`: opens browser page, sets content, emulates print, returns PDF buffer.
- `withTimeout(promise, timeoutMs, message)`: rejects long-running work with 504-status error.
- `getBrowser()`: reuses browser launch promise.
- `launchBrowser()`: launches Vercel Chromium/puppeteer-core or local Puppeteer.
- `renderPlanExportHtml(planRecord, options)`: builds complete export HTML.
- `renderMealCard(meal, mealIndex)`: renders one meal card.
- `renderFoodRow(item)`: renders one food row.
- `renderSummary(targets, actual)`: renders daily total summary.
- `renderMacroMetric(key, targets, actual)`: renders one macro bar metric.
- `exportCss()`: returns PDF-specific CSS overrides.
- `totalsForMeals(meals)`: sums meal totals.
- `totalsForItems(items)`: sums item totals.
- `totalsForItem(item)`: calculates item totals from quantity and food rates.
- `zeroTotals()`: returns zero macro totals.
- `normalizeTotals(totals)`: coerces totals to finite numbers.
- `addTotals(left, right)`: adds totals.
- `foodMedia(food)`: returns image/icon HTML for food.
- `imageDataUrl(filePath)`: embeds image file as data URL.
- `foodIcon(food)`: chooses fallback icon and tone by food name/category rules.
- `iconSvg(name, size)`: returns inline SVG string.
- `mealIconName(tag)`: maps meal tag to icon.
- `mealTypeKey(tag)`: normalizes meal tag for CSS/data attributes.
- `formatNumber(value)`: rounds/prints numbers.
- `clamp(value, min, max)`: clamps number.

- `pdfFilename(plan)`: generates safe lowercase kebab-case filename.
- `escapeHtml(value)`: escapes HTML-sensitive characters.

Stay or split: Split rendering helpers from browser lifecycle if export templates grow.

Scalability tips: A shared browser can become a bottleneck or leak memory. Add browser recycling, concurrency limits, and async export jobs.

public/index.html

What it does: Public landing/welcome page and redirect logic.

Input: Browser session state via `/api/auth/me` and click on Generate CTA.

Output/exports: No module exports. Renders landing page, redirects logged-in users to dashboard, directs unauthenticated users to login.

Functions/classes/components/helpers:

- Inline auth check IIFE: redirects logged-in users to `/dashboard`.
- Generate CTA click handler: checks auth and routes to dashboard or login.
- Inline nav greeting IIFE: replaces nav with greeting, dashboard link, logout button when logged in.
- `escapeHtml(v)`: inline helper to escape greeting.

Stay or split: Keep as a page, but move inline JS into `public/js/index.js` for maintainability.

Scalability tips: Static page is cheap. Avoid repeated `/api/auth/me` calls by consolidating the three inline auth checks.

public/login.html

What it does: Login form page shell.

Input: User email/password, Google auth button, forgot-password email.

Output/exports: Loads `public/js/auth.js`; no exports.

Functions/classes/components/helpers: HTML form/components only; behavior is in `public/js/auth.js`.

Stay or split: Keep as one file.

Scalability tips: Static page is fine. Auth rate limiting must happen server/Firebase side.

public/register.html

What it does: Registration form page shell.

Input: First name, last name, email, password, confirm password, Google auth button.

Output/exports: Loads `public/js/auth.js`; no exports.

Functions/classes/components/helpers: HTML form/components only; behavior is in `public/js/auth.js`.

Stay or split: Keep as one file.

Scalability tips: Static page is fine. Client password strength is helpful but not a server security boundary.

public/planner.html

What it does: Planner page shell, form fields, result containers, and HTML templates.

Input: User body data, plan setup fields, preferences, save/customer controls, generated/loaded plan state.

Output/exports: Loads `public/js/app.js`; no exports.

Functions/classes/components/helpers: HTML templates for summary and meal cards; behavior is in `public/js/app.js`.

Stay or split: Keep the HTML page, but reduce template complexity once a component/build system is introduced.

Scalability tips: Static HTML is cheap; bundle size and client-side DOM work in `app.js` are the main concerns.

public/dashboard.html

What it does: Dashboard page shell with views for home, customers, customer detail/edit, plans, filters, menus, and quick actions.

Input: Authenticated dashboard JSON and user interactions.

Output/exports: Loads `public/js/dashboard.js`; no exports.

Functions/classes/components/helpers: HTML view containers and controls; behavior is in `public/js/dashboard.js`.

Stay or split: Keep as page shell. Move repeated SVG/nav markup into shared frontend templates if the static approach continues.

Scalability tips: Dashboard payload and DOM rendering need pagination/virtualization for large plan/customer counts.

public/customer.html

What it does: Standalone customer detail page shell.

Input: Customer ID from URL path and authenticated session.

Output/exports: Loads `public/js/customer.js`; no exports.

Functions/classes/components/helpers: HTML shell only.

Stay or split: Keep as one file, though it overlaps with dashboard customer-detail behavior.

Scalability tips: Consider consolidating with dashboard route to reduce duplicate frontend behavior.

public/explorer.html

What it does: Folder/plan explorer page with inline JavaScript.

Input: Authenticated session, `folderId` query parameter, folder/plan UI actions.

Output/exports: No module exports. Calls folder and plan APIs.

Functions/classes/components/helpers:

- `load()`: authenticates, loads folder/root contents, and renders page.
- `renderContents({ subfolders, plans })`: renders folders and plans.
- `renderBreadcrumb()`: loads and displays breadcrumb.
- `makeLink(href, text)`: creates anchor element.

- `plannerHref(planId)`: builds planner edit URL.
- `updateActions()`: updates new-plan action links.
- `positionMenu(e)`: positions context menu.
- `showFolderMenu(e, folderId, folderName)`: shows folder actions.
- `showPlanMenu(e, planId, planName)`: shows plan actions.
- `openFolderModal(folderId, existingName)`: opens create/rename modal.
- `openDupModal(planId, planName)`: opens duplicate modal.
- `renderRootTreeItem(container)`: renders root tree node.
- `renderTree(nodes, container, depth)`: recursively renders folder tree.
- `esc(str)`: escapes HTML text.
- `svg(body)`: wraps SVG body string.

Stay or split: Split inline JavaScript into `public/js/explorer.js`.

Scalability tips: Folder tree rendering fetches all folders and recursively renders them; add lazy loading for large folder hierarchies.

public/js/auth.js

What it does: Implements browser authentication flow for login/register pages.

Input: DOM forms, Firebase public config, Firebase browser SDK, user credentials, Google auth responses.

Output/exports: No module exports. Creates Firebase sessions and redirects to dashboard.

Functions/classes/components/helpers:

- `initAuthPage()`: initializes page, checks existing app session, loads Firebase config/SDK, binds events.
- `bindEvents()`: attaches form and button handlers.
- `handleEmailSubmit(event)`: routes submit to signup or login.
- `handleSignup()`: validates, creates Firebase user, updates profile, sends verification.
- `handleLogin()`: signs in with Firebase email/password, requires verification, creates server session.
- `handleGoogle()`: signs in with Google popup or redirect fallback.
- `handleGoogleRedirectResult()`: handles redirect-return credential.
- `finishGoogleSignIn(user)`: creates server session and transitions to dashboard.
- `userHasProvider(user, providerId)`: checks linked Firebase provider.
- `waitForAuthUser(timeoutMs)`: waits for Firebase auth state.
- `createServerSession(user)`: sends Firebase ID token/profile to `/api/auth/session`.
- `resendVerification()`: sends verification email to pending user.
- `showForgotPassword()`: shows reset form.
- `hideForgotPassword()`: hides reset form.
- `handleForgotPassword(event)`: sends reset email.
- `loadFirebaseSdk()`: dynamically imports Firebase app/auth modules from Google CDN.
- `getLoginData()`: reads login form.
- `getRegisterData()`: reads register form.
- `validateSignup(data)`: validates registration fields and password.
- `bindPasswordStrength()`: connects password input to `zxcvbn`.
- `loadZxcvbn()`: loads `/js/zxcvbn.browser.js`.

- `authErrorMessage(err)`: maps Firebase/server errors to user messages.
- `setMessage(text, tone)`: updates auth message.
- `showAuthTransition(args)`: shows transition overlay/state.
- `hideAuthTransition()`: hides transition state.
- `isForgotPasswordVisible()`: checks reset form visibility.
- `setLoading(loading, label)`: toggles form loading state.
- `setDisabled(disabled)`: disables/enables controls.
- `setResendVisible(visible)`: toggles verification resend button.
- `clearHints()`: clears field hints.
- `escapeHtml(value)`: escapes text for UI.

Stay or split: Split Firebase SDK/session helpers from form UI once auth features grow.

Scalability tips: Firebase handles credential scale; app server should rate-limit `/api/auth/session` and monitor failed sessions.

public/js/dashboard.js

What it does: Implements dashboard SPA-style hash routing, rendering, filtering, CRUD actions, menus, and PDF downloads.

Input: `/api/dashboard`, `customer/plan` APIs, hash route, DOM events.

Output/exports: No module exports. Mutates DOM and navigates/downloads.

Functions/classes/components/helpers:

- `iconSvg(name, size)`: returns inline SVG icons.
- `escapeHtml(value)`: escapes text.
- `titleCase(value)`: formats identifiers.
- `goalLabel(goal)`: maps goal/status identifiers to labels.
- `formatRelativeTime(value)`: returns relative time labels.
- `initials(name)`: returns initials.
- `planHref(plan)`: builds planner edit URL.
- `planExportHref(plan)`: builds PDF export URL.
- `folderBreadcrumb(plan)`: renders folder path text.
- `pdfDownloadName(planName)`: builds local download filename.
- `customerGoalKey(customer)`: determines active/customer goal key; no current local caller found.
- `planGoalKey(plan)`: determines plan goal key.
- `matchesSearch(values, term)`: filters text.
- `countBy(items, keyFn)`: groups counts.
- `emptyState(label, detail)`: renders empty-state HTML.
- `goalTone(goal)`: maps goal to visual tone.
- `goalTag(goal)`: renders goal tag.
- `statusPill(active)`: renders active badge; no current local caller found.
- `currentPlanChip(active)`: renders current-plan chip.
- `customerDetailStats(customer)`: renders stat cards.
- `planRow(plan, options)`: renders plan row/card/table item.
- `customerRow(customer)`: renders customer row/card.

- `renderGoalBar(barId, legendId, counts)`: renders goal distribution bar.
- `filterChip(key, label, count, active)`: renders filter chip.
- `renderFilterChips(containerId, counts, total, activeKey, order)`: renders filter chips.
- `renderStats()`: updates dashboard stats.
- `sortByNewestCreated(plans)`: sorts plans.
- `renderHome()`: renders home view.
- `renderCustomersPage()`: renders customers view.
- `renderPlansPage()`: renders plans view.
- `loadCustomerPlans(customerId)`: fetches plans for customer.
- `renderCustomerDetail(customerId)`: renders customer detail route.
- `customerFormPayload(form)`: reads customer form body.
- `setCustomerFormValues(form, customer)`: populates edit form.
- `renderCustomerEdit(customerId)`: renders edit form.
- `setActiveNav(route)`: updates sidebar active state.
- `setMobileNavOpen(open)`: controls mobile drawer.
- `showPage(pageId, route)`: switches visible dashboard view.
- `parseHash()`: parses hash route.
- `renderRoute()`: routes based on hash.
- `ensureDashboardMenu()`: creates shared context menu.
- `hideDashboardMenu()`: hides menu.
- `positionDashboardMenu(button)`: positions menu.
- `exportHrefWithClientName(exportHref, hasCustomer)`: optionally asks client name and appends query.
- `downloadPlanPdf(exportHref, planName, options)`: downloads PDF via fetch/blob.
- `refreshDashboard()`: fetches `/api/dashboard?limit=100` and renders.
- `submitNewCustomer(form)`: posts customer create.
- `submitEditCustomer(form)`: sends customer update.
- `showPlanMenu(button)`: displays plan action menu.
- `showCustomerMenu(button)`: displays customer action menu.
- `installStaticIcons()`: hydrates icon placeholders.
- `initNav()`: authenticates and fills user nav.
- `bindEvents()`: binds dashboard UI events.

Stay or split: Split into API client, route renderer, components, and menu/actions modules.

Scalability tips: Current dashboard renders full result sets in DOM. Add server pagination and incremental rendering.

public/js/customer.js

What it does: Implements standalone customer-detail page.

Input: Customer ID from path, `/api/auth/me`, `/api/customers/:id/plans`, plan delete/duplicate actions.

Output/exports: No module exports. Renders nav/customer plan list and performs navigation.

Functions/classes/components/helpers:

- `iconSvg(name, size)`: returns inline SVG icons.

- `escapeHtml(value)`: escapes text.
- `formatRelativeTime(value)`: renders relative dates.
- `customerIdFromPath()`: extracts customer ID from URL.
- `planHref(plan)`: builds planner edit URL.
- `initNav()`: authenticates user and binds logout.
- `loadCustomer()`: loads customer plans and renders header/list.
- `renderPlans(plans)`: renders assigned plan cards and binds delete/duplicate.

Stay or split: Keep if page remains tiny. Consider removing if dashboard customer detail replaces it.

Scalability tips: Add pagination for customer plans.

public/js/app.js

What it does: Implements the planner UI, plan generation, plan editing, meal rendering, food swaps, produce cycling, autosave, PDF export, customer picker, preferences, and form synchronization.

Input: Planner form DOM, URL query params, `/api/*` responses, user clicks/searches/edit actions.

Output/exports: No module exports. Renders planner state, saves plans, downloads PDFs, navigates.

Functions/classes/components/helpers:

- `iconSvg, mealIconName, mealTypeKey, foodIcon, foodIconUrl, preloadFoodImage, preloadFoodImagesFromItems, foodFromPreferenceOption, setFoodMedia, foodMediaPlaceholder`: icon/image helpers.
- `produceGroup`: identifies fruit/vegetable produce.
- `readJsonResponse`: robust JSON response parsing.
- `generateAndRender`: posts generation request and renders result.
- `readForm, readPreGenerationPlanName, validatePreGenerationSaveDetails, preGenerationSavePayload, markProfileFieldTouched`: form/save-detail readers.
- `switchPlannerView, setInputsExpanded, updateSubmitIdleLabel, syncInputSummary, goalLabel, titleCase`: view and summary helpers.
- `loadAllFoods, ensureFoodsLoaded, ensurePreferenceOptionsLoaded, scheduleReferenceDataLoad`: reference-data loading.
- `savedPlanLoadError, showSavedPlanSkeleton, loadPlanForEdit, populateFormFromInput`: edit-plan loading.
- `mealOptionKey, mealItemsSignature, uniqueMealOptions, restoredMealOptionIndex, currentMealOptionIndex, persistCurrentMealOption`: meal option state helpers.
- `renderPlan, schedulePdfExport, waitForPdfExportAssets, isImpossiblePlan, renderPlanNotice`: main plan rendering and export readiness.
- `renderSummary, dailyRangeNoteHtml, refreshRedFlags`: daily summary and validation UI.
- `renderMealCard, setMealAiMode, refreshMealCustomizationControls, refreshMealCardHeader, mealCardMetaText, mealRangeNoteHtml, mealDisplayRanges, formatRangeValue`: meal card rendering/control helpers.
- `renderFoodList, foodRowKey, syncPendingAddLayer, foodRowSignature, updateFoodRow, renderPendingAddRow, setRowActions, setProduceCycleControl, renderFoodItem`: food row rendering/update helpers.
- `normalizeMealOption, mealOptionFitsTarget`: meal-option validation.
- `addTotals`: macro addition helper; no current local caller found.
- `dailyMetricFitsTarget, dailyMetricBounds, formatAllowedRange`: daily validation/range helpers.
- `normalizeStateItem, mealActionItems, customFoodPayload`: planner state/API payload helpers.

- `handleTryAnotherMeal`, `readyMealOptions`, `handleCycleMealOption`, `nextDaySafeMealOptionIndex`, `applyReadyMealOption`, `refreshMealCycleButtons`: ready-meal option cycling.
- `showAddFoodAction`, `focusPendingAddRow`, `removePendingAddRow`, `attemptInlineAddFood`: add-food workflow.
- `showRemoveFoodAction`, `createDeleteUndoContext`, `mealItemsUndoSignature`, `ensureDeleteUndoHost`, `showDeleteUndoToast`, `dismissDeleteUndoToast`, `restoreDeletedFood`: delete/undo workflow.
- `showSwapFoodAction`, `uniqueFoods`, `attemptSwapFood`: swap workflow.
- `handleCycleProduceSwap`, `createProduceSwapCache`, `ensureProduceSwapCache`, `resetProduceSwapCache`, `scheduleProduceSwapPreload`, `preloadProduceSwapOptions`, `loadProduceSwapOptionsForItem`, `produceSwapCacheKey`, `produceSwapCacheKeyForItems`, `fetchProduceSwapEntryForItems`, `mealItemsCacheSignature`, `userPreferenceCacheSignature`, `roundForCache`, `usableProduceOptions`, `nextCachedProduceSwapOption`, `takeProduceOptionFromEntry`, `applyProduceSwapOption`: produce-swap caching and cycling.
- `handleDeterministicRebalance`: calls rebalance workflow; no current local caller found.
- `attemptGuidedRebalance`, `editFailureClass`, `showProposal`, `showDeclineRetry`, `applyProposal`, `applyMealItems`, `pulseFoodRow`: deterministic rebalance/proposal workflow.
- `reserveSpaceForSaveBar`, `actionPanel`, `showActionMessage`, `resetActionPanel`, `clearFeedbackTimer`, `showActionFeedback`: UI feedback helpers.
- `renderFoodSearchResults`, `scoreFoodForMealSearch`, `foodAllowedForCurrentPreferences`, `foodFromCustom`: food search and custom-food helpers.
- `mergeSolvedQuantities`: applies solved quantities to attempted items.
- `itemFromGuidedProposal`: converts proposal item to state item; no current local caller found.
- `formatMacroLine`, `formatPortion`: display helpers.
- `compactRejectedProposal`: compacts rejected proposal; no current local caller found.
- `planCreateUrl`, `planExportUrl`, `requestPdfClientName`, `planWillHaveCustomer`, `startPlanExport`: plan URL/export helpers.
- `createGeneratedPlanRecord`, `savePlanRecord`, `setAutosaveStatus`, `flushAutosave`, `scheduleAutosave`, `deleteCurrentPlan`, `showEditBar`, `showInitialCreationBar`, `updatePendingPlan`, `showPlanSaveBar`: save/autosave/edit bar behavior.
- `bindCustomerPicker`, `refreshCustomerMatches`, `renderCustomerExactMatch`, `renderCustomerResults`, `loadCustomerForPlanning`, `selectCustomer`, `initializeCustomerPickerFromPlan`, `applyCustomerProfileToForm`, `setFormValue`, `buildCustomerPayload`: customer picker/profile sync.
- `buildPlanData`, `resetChat`, `getUserPreferences`, `computeTotals`, `itemTotals`, `clampGrams`: plan data and math helpers.
- `loadPreferenceOptions`, `hydrateAvoidFoodPreferences`, `setupPreferencePicker`, `renderTokens`, `addPreference`, `renderSuggestions`, `hideSuggestions`, `optionById`, `scoreOption`, `normalizeText`: preference picker behavior.
- `setLoading`, `syncRamadanControls`, `formatNumber`, `escapeHtml`: global UI helpers.
- Inline arrow helpers detected by static scan include small local functions such as `plannerCtx` setup, fallback image loading, field setters, keyed row reuse, and initialization callbacks.

Stay or split: Split this file urgently for maintainability. Suggested modules: API client, form state, renderers, meal editor, produce swaps, autosave, customer picker, preferences, and utilities.

Scalability tips: Large monolithic browser JS increases parse/maintenance cost. Add bundling/code-splitting and reduce repeated DOM work as features grow.

public/css/styles.css

What it does: Provides all frontend styling for landing, auth, planner, dashboard, customer, explorer, and PDF export reuse.

Input: HTML class names and state classes from public pages and `planPdfService`.

Output/exports: CSS rules; no JS exports.

Functions/classes/components/helpers: Not applicable.

Stay or split: Split into base, auth, planner, dashboard, explorer, components, and PDF/export styles.

Scalability tips: Large global CSS risks accidental regressions. Add naming conventions, visual regression tests, and remove unused styles after page inventory.

used_food_repository/foods.json

What it does: Runtime nutrition food database used by `foodRepository` and plan generation.

Input: Static JSON array with 108 foods.

Output/exports: Parsed into normalized food objects by `loadFoods()`.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file while small. Move to database/admin-managed source if food data changes frequently.

Scalability tips: Validate schema in CI. Store icon path availability in data or a build manifest.

ready_meals/meals.json

What it does: Runtime ready-meal bundle database used by `readyMealRepository` and plan generation.

Input: Static JSON object with metadata and `meal_bundles` for breakfast, lunch, dinner, and snack.

Output/exports: Parsed into flattened ready meal bundle objects.

Functions/classes/components/helpers: None.

Stay or split: Keep as one file while small. Split by meal tag or migrate to database if content becomes editable.

Scalability tips: Add data validation and coverage tests for diet/restriction combinations.

icons/*.png

What it does: Food icon assets served at `/food-icons/*` and embedded as data URLs in PDF export.

Input: Static PNG files matching food IDs.

Output/exports: Public image URLs and PDF embedded images.

Functions/classes/components/helpers: None.

Stay or split: Keep directory structure. Do not document each image separately unless asset provenance/licensing is required.

Scalability tips: Serve through CDN with immutable hashed filenames. Current filename mapping has 103 icons for 108 foods.

test-solver.js

What it does: Standalone smoke test for deterministic ready-meal generation.

Input: Hard-coded sample planner input.

Output/exports: No exports. Logs PASS/FAIL and exits with code 1 on failure.

Functions/classes/components/helpers:

- `assert(condition, msg)`: logs pass/fail and increments counters.

Stay or split: Keep only if updated. It currently appears stale because it expects 2-meal plans to be rejected while `planGenerator.normalizeInput` allows 2-meal plans.

Scalability tips: Convert into an npm test suite with multiple fixtures and CI integration.

E. API Contract Summary

Public endpoints:

- `GET /api/health` -> `{ status: 'ok' }`.
- `GET /api/foods` -> `{ foods }`.
- `GET /api/preferences` -> preference options object.
- `GET /api/auth/firebase-config` -> Firebase public config or missing-key error.

Authenticated endpoints:

- `POST /api/auth/session` with `{ idToken, profile? }` -> `{ user }`.
- `POST /api/auth/logout` -> `{ message }`.
- `GET /api/auth/me` -> `{ user }`.
- `DELETE /api/auth/me` -> `{ ok: true }`.
- `GET /api/dashboard?customerQuery&generalQuery&limit` -> dashboard summary.
- `GET/POST/PUT/DELETE /api/customers...` -> customer and plan summary payloads.
- `GET/POST/PATCH/DELETE /api/folders...` -> folder tree/content and folder plan-save payloads.
- `POST /api/generate-plan` -> generated plan with targets, meals, diagnostics, warnings/errors if any.
- `POST /api/rebalance-meal` -> `{ success, items?, totals?, mealValidation? }`.
- `POST /api/produce-swap-options` -> `{ group, options }`.
- `POST/GET/PUT/DELETE /api/plans...` -> plan CRUD, duplicate, active, and PDF export.

Common error shape: Most handlers return `{ error: string }`; some auth errors also include `{ code }`; Firebase config errors include `{ missing }`.

F. Unused, Uncertain, and Legacy Code Findings

Probably legacy/not runtime:

- `pinch-dashboard_4.html`: standalone prototype/dashboard mockup with no detected route/import/deploy reference.
- `pp.txt`: historical implementation prompt, not runtime.
- `src/Pinch_UI_Polish_Report.docx`: binary report artifact, not runtime.
- `USDA_database/database.zip`: source/archive data, not runtime.

Probably stale:

- `test-solver.js`: fails current behavior because it expects 2-meal plans to be rejected while current code allows 2-meal generation.

Probably unused exports or helpers, pending tests:

- `src/repositories/planRepository.js`: `getPlansByFolder`.
- `src/repositories/customerRepository.js`: `listCustomersWithPlanSummary`.

- `src/repositories/dashboardRepository.js`: `folderPathFor`.
- `src/repositories/userRepository.js`: `findUserByFirebaseUid`, `findUserByEmail`, `updateLastLogin`, `incrementTokenVersion`.
- `src/services/nutritionService.js`: several exported helper functions are only used internally or not currently called by app code; keep until tests clarify public helper expectations.
- `public/js/app.js`: `addTotals`, `handleDeterministicRebalance`, `itemFromGuidedProposal`, `compactRejectedProposal` had no local call hits in static scan.
- `public/js/dashboard.js`: `customerGoalKey`, `statusPill` had no local call hits in static scan.

Risky to delete without more testing:

- Any route/controller/repository function registered by Express routes.
- `api/plan-export.js`, because Vercel rewrites depend on it.
- `icons/*.png`, because URLs are generated dynamically from food IDs.
- Nutrition helpers exported from `nutritionService`, because they may be used by future tests or external scripts.

G. Scalability, Reliability, Security, and Maintainability Risks

Scalability:

- Synchronous generation and rebalance work runs in the request process.
- PDF export launches/uses headless Chromium in the request path.
- Sequelize pooling can exhaust PostgreSQL under serverless scale.
- Dashboard raw SQL uses recursive CTEs and JSONB extraction; indexes are not visible in repo.
- Folder breadcrumb uses one query per depth.
- Static JSON data is loaded in memory and is fine now, but not ideal for frequently changing large datasets.

Reliability:

- No migrations are present, so model/schema drift is hard to control.
- `getPlanById(... markOpened)` updates last-opened asynchronously and swallows errors.
- Deleting folders depends on database constraints not shown in this repo.
- `errorHandler` can expose raw 500 messages.

Security:

- Add rate limiting to auth/session/generation/PDF endpoints.
- Add request body schemas and stricter payload limits.
- Revisit `rejectUnauthorized: false` for DB SSL.
- Add CSRF strategy if cookie-authenticated state-changing routes remain same-site browser APIs.
- Add security headers.

Maintainability:

- `public/js/app.js`, `public/js/dashboard.js`, `public/css/styles.css`, and `src/services/planGenerator.js` are too large for comfortable long-term maintenance.
- Inline JavaScript in `public/index.html` and `public/explorer.html` makes reuse/testing harder.
- Duplicate PDF export paths should share one request handler.

H. Verification Performed

- Ran `npm run check`: passed JavaScript syntax checks for `src/**/*.js` and public JS files.
- Ran `node test-solver.js`: failed one stale expectation, 2-meal plans are rejected.
- Counted runtime data: `used_food_repository/foods.json` has 108 foods; `ready_meals/meals.json` has breakfast/lunch/dinner/snack bundles; `icons/` has 103 PNGs matching food IDs.